Step 5.4b – nginx logs to stor01 + one standard setup script · complete case

Project 2 – Acme Analytics Linux platform · Phase 5 · ticket #12 · builds on Step 5.4 (ticket #10) · Day 18, 13 Nov 2026 · ✔ solved

Phase 5 · Step 5.4b – nginx logs to stor01 + one standard setup script

Project 2 – Acme Analytics Linux platform · Day 18 · ticket #12 · builds on Step 5.4 (ticket #10)

GLPI · Ticket #12 · New

Title	**Shop errors are not on the central log server – and new servers have no log setup**
Requester	Sara (IT lead)
Priority	Medium
Servers	app01 (nginx), gw01 (not sending yet), stor01

"stor01 has app01's system logs, but not the **web shop's own logs** – who visited, and the shop's errors (ticket #9 case ①). And every time we add a server, someone has to set up logging by hand. Please send nginx's logs to stor01, and make **one script** that sets up central logging on any server."

The picture after 5.4b

app01 · nginx
own files stay:
`/var/log/nginx/access.log` , `error.log`
+ new: `/etc/nginx/conf.d/central-logs.conf`
→ also to syslog, tags `nginx_access` , `nginx_error`

gw01 · not sending yet
`enable-central-logs.sh` puts `90-forward.conf` in place → gw01 sends too

TCP 514
live, line by line
➜

stor01 · /var/log/remote/
app01/ nginx_access.log · nginx_error.log · sudo.log …
db01/ postgres.log …
gw01/ sshd.log · systemd.log … (new)

enable-central-logs.sh – checks what is installed, places one drop-in per program, tests, reloads. Never overwrites a file that exists.
always → rsyslog `90-forward.conf` · nginx? → `conf.d/central-logs.conf` · PostgreSQL? → `conf.d/central-logs.conf` · Docker? → `daemon.json`

What earlier steps give this one

Step 5.4: stor01 receives on 514, sorts per server and program
Day 17 notes: programs with own files need one setting (nginx `syslog:`)
Day 16: nginx on app01 · Phase 2: gw01 has internet (to download the script)
server-report.sh: the same "check what is installed" idea

New words

access log – one line per page visit · **error log** – nginx's problems
/dev/log – the "letterbox" of the system log; nginx drops its lines there
conf.d – nginx's drop-in folder, read inside its main settings
idempotent – run the script twice, nothing breaks (it skips what exists)

Plan

1. **A · nginx by hand** on app01: one drop-in file, `nginx -t` , reload, open a page and a missing page → lines on stor01
2. **B · The script** on gw01: download from GitHub, run → gw01 appears on stor01
3. **C · Run it again** on app01: everything SKIP – it never overwrites
4. **D · Close ticket #12**

How logs live and travel – read this first

How logs live and travel on a Linux server

Step 5.4b · app01 · one folder for all logs, one door into the system log, one road to stor01

/var/log/ – the main folder for all logs

```
/var/log/
├── journal/      journald's files – the ONLY thing journalctl shows
├── syslog        rsyslog's text copy (mixed)
├── auth.log      logins, sudo
├── nginx/        access.log · error.log
├── apache2/      access.log · error.log
├── postgresql/   postgresql-14-main.log
├── mysql/        error.log
└── remote/       only on stor01 – the central copies
```

blue = the system log (journald + rsyslog) · **orange** = a program's **own** files – journalctl does **not** show them; read with `sudo tail <file>` (Docker: `docker logs <name>`) · **green** = stor01's sorted copy

The road of one nginx line

Without the setting – the line stays in nginx's own file

nginx (a visit) ➜ /var/log/nginx/ (access.log · error.log) × journald never sees it → not on stor01

With the setting (conf.d/central-logs.conf) – the same line takes two roads at once

nginx (a visit) ➜ /var/log/nginx/ (own file stays)

nginx (a visit) ➜ /dev/log (a door (socket) always open, nothing waits) ➜ journald (stores in /var/log/journal/) ➜ rsyslog (sends live TCP 514) ➜ stor01 (remote/app01/ nginx_access.log nginx_error.log)

sudo, logger, sshd use the door **/dev/log** every day. The setting only tells **nginx** to use it too. From journald on, every line takes the same road: **journald** → **rsyslog** → **stor01** – and `journalctl -t nginx_access` now shows nginx's lines as well.

Final paragraph. Why 5.4b: after Step 5.4, stor01 had app01's **system** logs (sudo, systemd, sshd …) but **not the web shop's own logs** – who visited, and the shop's errors (ticket #9 case ①) – and every new server needed its logging set up by hand. So: (1) send nginx's logs to stor01, (2) one script that sets up central logging on any server.
Where logs live: `/var/log/` is the main folder for all logs; inside it each program has its own place: `journal/` (journald – the **only** thing `journalctl` shows), `syslog` and `auth.log` (rsyslog's text copies), and the programs' **own** files – `nginx/` (access.log = one line per visit, error.log = problems), `apache2/` , `postgresql/` , `mysql/` ; Docker keeps its own under `/var/lib/docker/containers/` . journalctl does not show the own files – read them with `sudo tail <file>` (Docker: `docker logs <name>`).
The road: without a setting, an nginx line stays in `/var/log/nginx/` – journald never sees it, so it never reaches stor01. With one drop-in file, `/etc/nginx/conf.d/central-logs.conf` (`access_log syslog:…,tag=nginx_access,nohostname;` and `error_log syslog:…,tag=nginx_error,nohostname;` – `nohostname` so journald reads our tag as the name), nginx writes each line **twice at the same moment**: into its own file (it stays) and into **/dev/log** – a **door** (socket) that journald always listens at, nothing waits there; sudo and logger use it every day, the setting only tells nginx to use it too. From there every line takes the same road: **journald** stores it (now `journalctl -t nginx_access` shows it) → **rsyslog** sends it live over TCP 514 → **stor01** sorts it into `remote/app01/nginx_access.log` and `nginx_error.log` .

The two curl uses: curl opens a web page from the command line, like a browser – it is not ping (ping: "is the server there?", curl: "does the **website** answer?"). **(1) Step 0 – does the web shop answer?** `curl … http://localhost/` → `200` = the page works. **(2) Step A2 – we act as a visitor on purpose**, because nginx only writes a line when someone visits: `http://localhost/` → 200 = a normal visit → one **access** line; `http://localhost/nothing-here` → 404 = a missing page → an access line **and** an **error** line – exactly the lines we then look for in journalctl and on stor01.
The script `enable-central-logs.sh` does this for the server it runs on: checks what is installed, places one drop-in per program (rsyslog, nginx, PostgreSQL, Docker), tests, reloads, and never overwrites an existing settings file (SKIP). Many servers from one place = Ansible, later.

Before you start

0 Is nginx still there? (installed on Day 16)

app01

Why: 5.4b sends nginx's own logs – nginx must be running. nginx was installed on Day 16 – nothing to install now. **curl** = a visitor from the command line: it opens the web page like a browser does. Here it only checks that the shop answers (`200` = page works) – it is **not** ping: ping only asks "is the server there?", curl asks "does the **website** answer?".

```
systemctl is-active nginx
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/
ls /etc/nginx/conf.d/
```

GOOD: `active` · `200` · conf.d empty. If inactive: `sudo systemctl start nginx`. If "not found": `sudo apt update && sudo apt install -y nginx`.

Part A · nginx by hand

A1 One drop-in file for nginx

app01

Why: nginx writes its access log (one line per visit) and error log (problems) only into `/var/log/nginx/` – rsyslog never sees them. A **second** `access_log` / `error_log` line also drops them into the system log; the own files stay.

```
sudo nano /etc/nginx/conf.d/central-logs.conf      # type 2 lines · save Ctrl+O, Enter · exit Ctrl+X

access_log syslog:server=unix:/dev/log,tag=nginx_access,nohostname;   # every visit → system log, name
nginx_access
error_log  syslog:server=unix:/dev/log,tag=nginx_error,nohostname;    # every problem → system log, name
nginx_error
```

Part of the line	Meaning
`access_log` / `error_log`	a second place for nginx's visit lines / problem lines (the first place, its own file, stays)
`syslog:server=unix:/dev/log`	send each line into the system log's door `/dev/log` – journald picks it up at once
`tag=nginx_access`	the **name** on the line – journalctl finds it with `-t nginx_access` ; on stor01 it becomes the file name
`nohostname`	leave the server name out of the line, so journald reads **our tag as the name** (without it, see A3)



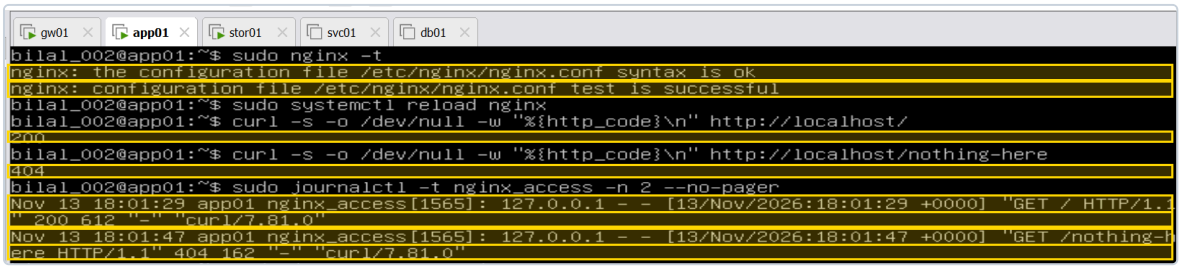
A2 Check, reload, make one visit and one error

app01

```
sudo nginx -t                                                    # check the settings BEFORE
loading them
sudo systemctl reload nginx                                      # load them — the shop keeps
running
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/       # act as a visitor: a normal page
→ 200 + one access line
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/nothing-here   # a missing page → 404 + an access
line AND an error line
sudo journalctl -t nginx_access -n 2 --no-pager                  # did the visit lines reach
journald under our name?
sudo journalctl -t nginx_error  -n 2 --no-pager                  # did the error line?
```

sudo is needed: nginx's lines are **system** lines (written by nginx, not by you) – without sudo, journalctl shows a user only their own lines.

Check	GOOD
nginx -t	**syntax is ok · test is successful**
curl	**200 · 404**
-t nginx_access	**… "GET / HTTP/1.1" 200 … and … "GET /nothing-here HTTP/1.1" 404 …**
-t nginx_error	"-- No entries --" in our lab – **not a fault**: Ubuntu's default site answers a missing page quietly (`try_files … =404`), so the 404 only shows in the **access** log. The error setting is in place; a real error would appear. To create one on purpose: `sudo mkdir /var/www/html/test-dir` → `curl …/test-dir/` → 403 + "directory index … is forbidden" → `sudo rmdir /var/www/html/test-dir` .

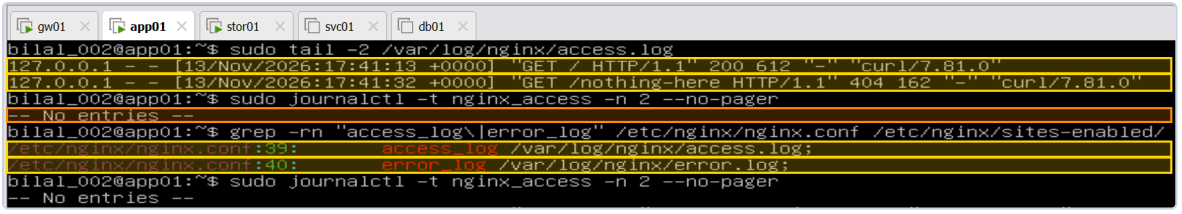


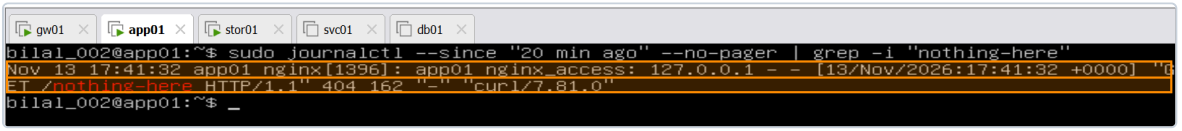
A3 What we found on the first try (without nohostname) – and why we added it

app01

What happened: nginx -t ok, 200 and 404 – but `journalctl -t nginx_access` said **"-- No entries --"**, even with sudo. So we checked step by step:

```
sudo tail -2 /var/log/nginx/access.log                                  # 1. did nginx log the visits at all? → yes, in its own file
grep -rn "access_log\|error_log" /etc/nginx/nginx.conf /etc/nginx/sites-enabled/   # 2. does a site file override our lines? → no
sudo journalctl --since "20 min ago" --no-pager | grep -i "nothing-here"  # 3. is the line in journald under ANOTHER name? → yes
```





Check	What it showed	Meaning
1 · own file	`"GET / HTTP/1.1" 200` · `"GET /nothing-here HTTP/1.1" 404`	nginx logs the visits – the problem is after it
2 · grep the settings	only nginx.conf lines 39, 40 (its own files)	no site file overrides our lines
3 · search journald	`app01 nginx[1396]: app01 nginx_access: … "GET /nothing-here …" 404`	the line **arrived** – but named **nginx**; our tag is hidden inside the text

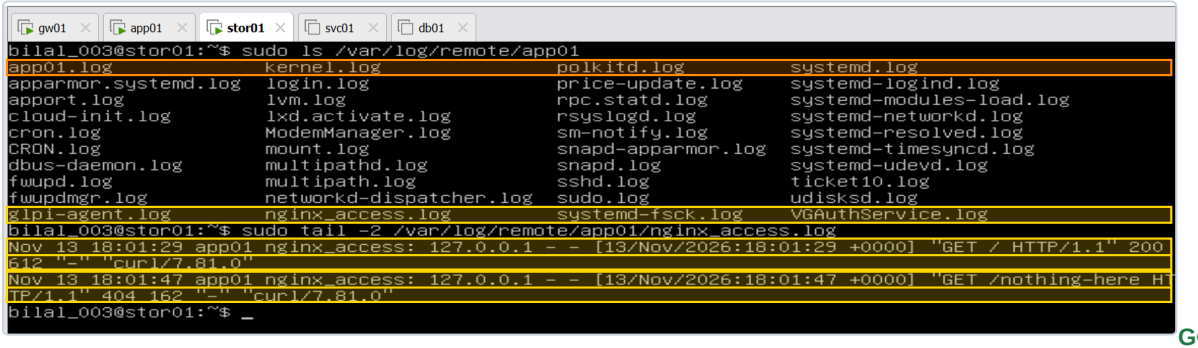
Why: nginx puts the **server name in front** of its line (`app01 nginx_access: …`) – the format for sending over a network. journald reads lines from `/dev/log` expecting the **name first**, finds `app01` , cannot use it, and falls back to the program's own name, **nginx**. So the search for `nginx_access` found nothing, and on stor01 the lines landed mixed in `nginx.log` . Like a letter that reaches the right house with the wrong name on the envelope.

Fix: add `,nohostname` to both lines (A1) → nginx leaves the server name out → our tag comes first → journald uses it as the name. Then A2 again: `-t nginx_access` and `-t nginx_error` find the lines.

A4 The lines arrive on stor01

stor01

```
sudo ls /var/log/remote/app01/                          # new files nginx_access.log and nginx_error.log?
sudo tail -2 /var/log/remote/app01/nginx_access.log     # the visit lines
sudo tail -2 /var/log/remote/app01/nginx_error.log      # the error line
```



GOOD: **nginx_access.log** with the same lines as in A2 → ticket #9 case ① evidence is now on stor01. The **app01.log** from the first try stays – it shows what went wrong.

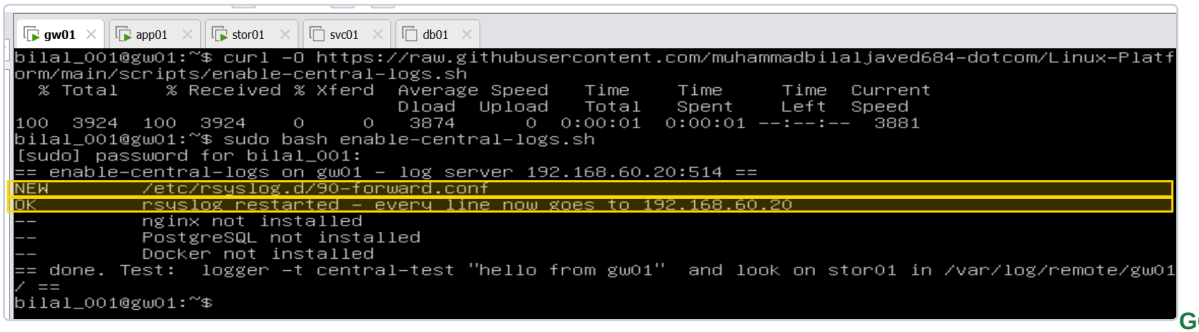
Part B · The script on a server that does not send yet

B1 Download and run enable-central-logs.sh

gw01

Why: setting up logging by hand on every new server is slow and easy to forget. The script checks what is installed, places **one drop-in per program**, tests it and reloads – and removes its own file again if a test fails. **It sets up only the server it runs on** – run on gw01, it sets up gw01; for app01 it must run on app01. gw01 is the test because it has internet (to download from GitHub) and does not send logs yet. The test line `central-test` shows that gw01's lines now arrive on stor01.

```
curl -O https://raw.githubusercontent.com/muhammadbilaljaved684-dotcom/Linux-Platform/main/scripts/enable-central-logs.sh
sudo bash enable-central-logs.sh
logger -t central-test "hello from gw01"
```



GOOD:

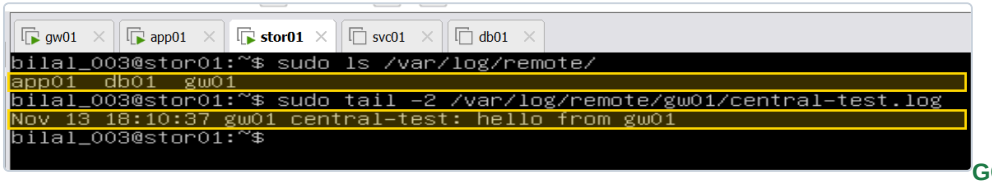
```
NEW         /etc/rsyslog.d/90-forward.conf
OK          rsyslog restarted – every line now goes to 192.168.60.20
--          nginx not installed
--          PostgreSQL not installed
--          Docker not installed
```

The script checks	and places	tests with
always (not on stor01 – it only receives)	`/etc/rsyslog.d/90-forward.conf`	`rsyslogd -N1` → restart rsyslog
nginx installed?	`/etc/nginx/conf.d/central-logs.conf`	`nginx -t` → reload nginx
PostgreSQL installed?	`/etc/postgresql/<ver>/main/conf.d/central-logs.conf`	reload PostgreSQL
Docker installed?	`/etc/docker/daemon.json` (only if missing)	restart Docker

B2 gw01 appears on stor01

stor01

```
sudo ls /var/log/remote/
sudo tail -2 /var/log/remote/gw01/central-test.log
```



GOOD: three folders **app01 · db01 · gw01** · `gw01 central-test: hello from gw01`.

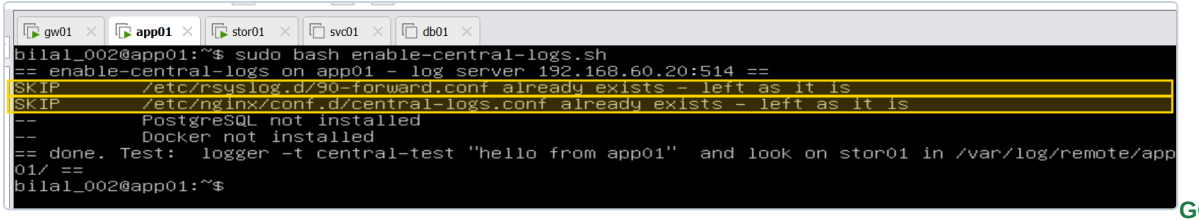
Part C · Run it again – nothing breaks

C1 Second run on app01

gw01 → app01

Why: a good setup script can run twice without damage ("idempotent"): it never overwrites a **settings file** that already exists – it says SKIP. (This is about settings files, not log lines.) That is also how to use it later: copy it to a server, run it there; if a new program was installed since, it adds only the missing file. Running it on **many servers from one place** is the job of Ansible – a later step.

```
# on gw01
scp enable-central-logs.sh bilal_002@app01:~
# on app01
sudo bash enable-central-logs.sh
```



GOOD:

```
SKIP       /etc/rsyslog.d/90-forward.conf already exists – left as it is
SKIP       /etc/nginx/conf.d/central-logs.conf already exists – left as it is
--         PostgreSQL not installed
--         Docker not installed
```

Part D · Close ticket #12

GLPI · Assistance › Tickets › #12

Title	**Shop errors are not on the central log server – and new servers have no log setup**
Status	**Solved**
Requester	Sara (IT lead) · Category: Linux › Logging · Priority: Medium
Servers	app01 (nginx), gw01 (new sender), stor01 (central log server)
Opened / solved	13 Nov 2026 · same day

✔ Solution

What was done: nginx on app01 now writes its access and error lines also into the system log, through one drop-in file `/etc/nginx/conf.d/central-logs.conf` (`syslog:server=unix:/dev/log,tag=nginx_access,nohostname` and the same for errors). nginx's own files stay. A new script, `enable-central-logs.sh` (on GitHub), sets up central logging on any server: it checks what is installed, places one drop-in per program, tests and reloads, and never overwrites existing settings.

Proof:

- app01: `journalctl -t nginx_access` shows the visits (GET / 200, GET /nothing-here 404) – and stor01 has them in `app01/nginx_access.log`.
- First try without `nohostname`: the lines arrived but under the wrong name (the server name stood in front) – fixed with one word.
- gw01: the script placed the rsyslog send file → stor01 now has a `gw01` folder with the test line.
- app01: the script run a second time → everything SKIP – nothing overwritten.

Note: the error log setting is in place; the test 404 produced no error line, because the shop answers a missing page quietly (try_files). A real error (e.g. a forbidden folder) would appear in `nginx_error.log`.

Final paragraph. nginx writes its **access log** (one line per page visit) and **error log** (problems) only into its own files in `/var/log/nginx/`, so they never reached stor01. One drop-in file, `/etc/nginx/conf.d/central-logs.conf`, adds a **second** access_log and error_log line: `syslog:server=unix:/dev/log` drops each line also into the system log, `tag=` names them `nginx_access` and `nginx_error`, and `nohostname` makes journald use that name (without it the lines arrived named `nginx` – A3); nginx's own files stay. `nginx -t` checks before the reload. A normal visit (200) and a missing page (404) prove it: the lines are in journald on app01 and on stor01 as `nginx_access.log` and `nginx_error.log` – ticket #9's case ① evidence, now central. The script `enable-central-logs.sh` does this for any server: it checks what is installed, places one drop-in per program (rsyslog, nginx, PostgreSQL, Docker), tests and reloads, and removes its own file if a test fails. On gw01 it made gw01 send – a new **gw01** folder on stor01. Run twice, everything is **SKIP** – it never overwrites, and on stor01 it skips the send file because stor01 only receives. Companies do the same job with an **Ansible role**.